



Proyecto IA BERT

Sistema de Clasificación de Currículums con Red Neuronal BERT

Django · PyTorch · Transformers · spaCy · Tesseract OCR | v1.0 | 2026-08-24

Contenido

1. [1. Instalación y Configuración](#)

2. [2. Arquitectura y Tecnologías](#)

3. [3. Red Neuronal BERT](#)

4. [4. Aplicación Web](#)
5. [5. Datos de Entrenamiento](#)

6. [6. Métricas y Gráficas](#)

7. [7. Despliegue en Render](#)

Guía de Instalación y Configuración

Pasos para instalar las dependencias e inicializar el sistema de clasificación de currículums.

Requisitos Previos

Requisito	Versión	Notas
Python	3.11 (recomendado)	<code>runtime.txt</code> define 3.11 para despliegue
pip	Última	Gestor de paquetes de Python
Tesseract OCR	5.x	Necesario solo si se procesan PDFs escaneados
Git	Opcional	Para clonar el repositorio

1. Estructura del Proyecto

```
Proyecto_IA_BERT/  
├── manage.py           # Punto de entrada Django  
├── proyecto_cv/        # Configuración del proyecto Django  
├── aplicacion_web/     # Aplicación web (vistas, URLs, templates)  
├── red_neuronal/       # Modelo BERT y NLP  
│   ├── src/           # Código fuente del modelo  
│   ├── models/        # Artefactos del modelo (generado en runtime)  
│   └── requirements.txt  
├── datos_entrenamiento/ # Dataset (training_data.csv)  
├── pdfs_temporales/    # PDFs subidos por usuarios (se eliminan solos)  
├── build.sh / render.yaml # Despliegue en Render  
└── docs/              # Documentación del proyecto
```

2. Instalar Dependencias

2.1 Crear entorno virtual

```
# Windows  
python -m venv venv  
venv\Scripts\activate  
  
# Linux/Mac  
python3 -m venv venv  
source venv/bin/activate
```

2.2 Dependencias del proyecto

```
# 1. Framework web  
pip install "Django>=4.0"  
  
# 2. Red neuronal (torch, transformers, spacy, OCR, etc.)  
cd red_neuronal  
pip install -r requirements.txt
```

Las dependencias principales son: `torch`, `transformers`, `pandas`, `scikit-learn`, `spacy`, `pytesseract`, `deep-translator`, `langdetect`, `pymupdf`, `Pillow`. Ver [02-arquitectura-tecnologias.md](#) para el rol de cada una.

2.3 Descargar modelo de spaCy

```
python -m spacy download en_core_web_sm
```

2.4 Instalar Tesseract OCR (solo PDFs escaneados)

Windows:

1. Descargar desde: <https://github.com/UB-Mannheim/tesseract/wiki>
2. Instalar y agregar al PATH del sistema
3. Opcional: instalar datos de idioma español (`tesseract-ocr-spa`)

Linux (Ubuntu):

```
sudo apt-get update && sudo apt-get install tesseract-ocr tesseract-ocr-spa
```

Mac (Homebrew):

```
brew install tesseract
```

3. Configurar el Proyecto Django

El proyecto ya viene configurado; estos pasos son de verificación:

3.1 settings.py (`proyecto_cv/settings.py`)

```
INSTALLED_APPS = [..., 'aplicacion_web']

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'pdfs_temporales'

# Ruta del núcleo de IA al PYTHONPATH
sys.path.insert(0, os.path.join(BASE_DIR, 'red_neuronal'))
```

3.2 Crear carpetas generadas en runtime

```
mkdir pdfs_temporales          # PDFs temporales de usuarios
mkdir red_neuronal/models      # Artefacto del modelo entrenado
```

4. Ejecutar el Proyecto

```
# Migraciones
python manage.py makemigrations
python manage.py migrate

# Servidor de desarrollo
python manage.py runserver
```

La aplicación queda disponible en: <http://127.0.0.1:8000/>

Nota PowerShell: si Django no encuentra el módulo de settings, ejecutar antes:

```
$env:DJANGO_SETTINGS_MODULE="proyecto_cv.settings"
```

5. Modelo BERT

Al iniciar, la aplicación verifica si existe `red_neuronal/models/bert_classifier_category.pt` :

1. **Si existe** → se carga directamente.
2. **Si NO existe** → se descarga automáticamente desde Google Drive usando `gdown` (implementado en `aplicacion_web/views.py`).

Descarga manual (alternativa)

Enlace: <https://drive.google.com/file/d/1dXUht0jrVIA8lqWTLEHPHXd-8kgRc1Um/view?usp=sharing>

Guardar como: `red_neuronal/models/bert_classifier_category.pt`

Entrenamiento local (opcional)

```
cd red_neuronal
python train_bert_classifier.py
```

Requiere el dataset en `datos_entrenamiento/1_resume_classification/training_data.csv` . Ver [05-datos-entrenamiento.md](#).

6. Verificar la Instalación

1. Abrir <http://127.0.0.1:8000/>
 2. Subir un PDF de currículum
 3. El sistema debe mostrar la categoría profesional predicha y su confianza
-

7. Solución de Problemas

ModuleNotFoundError: No module named 'src'

Agregar la ruta del núcleo de IA al `PYTHONPATH` (ya incluida en `settings.py`):

```
import sys, os
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.path.join(BASE_DIR, 'red_neuronal'))
```

TesseractNotFoundError

Configurar la ruta de Tesseract en `views.py`:

```
import pytesseract
pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\tesseract.exe'
```

No such file or directory: training_data.csv

Verificar que el dataset exista en `datos_entrenamiento/1_resume_classification/training_data.csv` .

8. Resumen de Comandos

```
venv\Scripts\activate          # Activar entorno (Windows)
pip install "Django>=4.0"
cd red_neuronal && pip install -r requirements.txt
python -m spacy download en_core_web_sm
python manage.py makemigrations && python manage.py migrate
python manage.py runserver
```

Arquitectura y Tecnologías

Descripción técnica de las tecnologías del proyecto, su rol en el pipeline y la justificación de cada elección.

1. Pipeline Completo del Sistema

```
Usuario sube PDF
↓
Django recibe petición (views.py)
↓
PyMuPDF (PDF digital) o Tesseract OCR (PDF escaneado)
↓
GoogleTranslator (traducción al inglés)
↓
spaCy (NER, lematización, extracción de secciones)
↓
TextCleaner (limpieza de texto)
↓
BERT tokenizer (convertir texto a números)
↓
Modelo BERT (predicción de categoría)
↓
Django renderiza resultado en HTML
```



Usuario ve la predicción

2. Django (Framework Web)

Framework web full-stack de Python que maneja toda la **interfaz web**:

- Servidor web que recibe peticiones de los usuarios
- Rutas (URLs) hacia las vistas
- Plantillas HTML con los resultados
- Gestión de archivos subidos (PDFs)
- Base de datos (SQLite)

Componentes usados

Componente	Función en el proyecto
<code>django.shortcuts.render</code>	Renderiza plantillas HTML con datos
<code>django.core.files.storage</code>	Guarda archivos subidos (PDFs)
<code>django.conf.settings</code>	Configuración global
<code>django.urls.path</code>	Define rutas/URLs
<code>django.views</code>	Funciones que procesan peticiones

¿Por qué Django y no Flask?

Característica	Flask	Django
Tipo	Micro-framework	Full-stack
Base de datos	No incluida	ORM incluido
Autenticación	Manual	Incluida
Estructura	Flexible	Opinada (opinionated)

Característica	Flask	Django
Tamaño de proyecto	Pequeño-mediano	Grande

Para este proyecto Django aporta más estructura lista, mejor manejo de seguridad de archivos subidos y mejor integración con templates.

3. Dependencias de IA y Procesamiento

Deep Learning y Modelos de Lenguaje

torch (PyTorch) — Framework de deep learning de Meta. Permite entrenar redes neuronales mediante diferenciación automática y retropropagación. Usado para definir, entrenar y evaluar el modelo BERT.

transformers (Hugging Face) — Librería con arquitecturas pre-entrenadas de última generación (BERT, RoBERTa, DistilBERT). Incluye tokenizadores, configuraciones y métodos de fine-tuning. Es el estándar de la industria para NLP.

Ciencia de Datos

pandas — Manipulación y análisis de datos con DataFrames optimizados. Carga y preprocesa el dataset de currículums.

numpy — Computación numérica: arrays multidimensionales y funciones matemáticas de alto rendimiento.

scikit-learn — ML clásico: métricas (accuracy, F1-score), validación cruzada, split de datos y evaluación.

NLP y Texto

spaCy — Framework industrial de NLP:

- NER: reconocimiento de entidades (personas, organizaciones, fechas)
- POS tagging: categorías gramaticales
- Lemmatización: reducción de palabras a su raíz
- Dependency parsing: análisis sintáctico

Modelo usado: `en_core_web_sm`.

deep-translator — Traducción automática del español al inglés antes de procesar (el modelo está entrenado en inglés).

langdetect — Detección automática de idioma (algoritmo n-gram); evita traducir textos ya en inglés.

Extracción de Texto de PDFs

pymupdf (fitz) — Extrae texto de PDFs digitales (con texto embebido) y convierte páginas a imágenes para OCR.

pytesseract — Wrapper de Python para Tesseract OCR (motor de Google). Convierte imágenes a texto; requiere Tesseract instalado en el sistema.

Pillow — Procesamiento de imágenes: convierte páginas PDF a imágenes antes de pasarlas a Tesseract.

Utilidades

python-dotenv — Variables de entorno desde `.env`, separando configuración sensible del código.

tqdm — Barras de progreso durante el entrenamiento.

4. Resumen: Rol por Tecnología

Tecnología	Propósito
Django	Servidor web y estructura
PyMuPDF	Leer PDFs digitales
Tesseract + pytesseract	Leer PDFs escaneados (OCR)
Pillow	Manipular imágenes
GoogleTranslator	Traducción automática

Tecnología	Propósito
langdetect	Detección de idioma
spaCy	Procesamiento de lenguaje natural
BERT + transformers	Modelo de machine learning
torch	Framework de deep learning
pandas / scikit-learn	Manejo y evaluación del dataset

Cada tecnología cumple un rol específico en el pipeline de extremo a extremo: extraer → traducir → limpiar → analizar → clasificar.

DOCUMENTACIÓN TÉCNICA DE LA RED NEURONAL BERT

Sistema de Clasificación de Currículums

1. INTRODUCCIÓN Y CONTEXTO

1.1 ¿Qué es una Red Neuronal?

Una **red neuronal artificial** es un modelo computacional inspirado en cómo funcionan las neuronas del cerebro biológico. Está compuesta por:

- **Neuronas** (nodos): unidades que procesan información
- **Capas**: grupos de neuronas que operan en paralelo
- Capa de entrada: recibe los datos
- Capas ocultas: procesan la información
- Capa de salida: produce el resultado

- **Pesos y sesgos:** parámetros que se ajustan durante el entrenamiento

Entrada → [Capa 1] → [Capa 2] → ... → [Capa Salida] → Predicción

1.2 ¿Por qué usamos una red neuronal para este proyecto?

Problema a resolver: Clasificar automáticamente currículums en 42 categorías profesionales basándose en su contenido.

Por qué no usar reglas simples?

- Los currículums varían enormemente en formato y contenido
- Las categorías no son obvias (un "Python developer" podría estar en "Technology" o "Software")
- El lenguaje natural es complejo y ambiguo

Por qué una red neuronal?

- Aprende patrones complejos directamente de los datos
- Generaliza a ejemplos no vistos
- Puede capturar relaciones semánticas sutiles
- El texto de un CV tiene muchas características implícitas

2. ELECCIÓN DE LA ARQUITECTURA: BERT

2.1 ¿Qué es BERT?

BERT (Bidirectional Encoder Representations from Transformers) es una arquitectura de red neuronal desarrollada por Google en 2018. Es un **Transformer** - una arquitectura que revolucionó el procesamiento del lenguaje natural.

¿Por qué BERT y no otra arquitectura?

Arquitectura	Ventajas	Desventajas	¿Por qué no?
BERT	Pre-entrenado, bidirectional,state-of-the-art	Más pesado	 Elegido

Arquitectura	Ventajas	Desventajas	¿Por qué no?
LSTM/RNN	Secuencial, bueno para secuencias	Lento, no captura contexto largo	No escala bien
CNN (texto)	Rápido	No captura relaciones lejanas	No óptimo para NLP
Red simple	Rápido	Poco poder expresivo	No suficiente

Razones específicas de la elección:

1. **Pre-entrenamiento:** BERT ya fue entrenado con grandes cantidades de texto en inglés, lo que significa que ya "entiende" el lenguaje. Solo necesitamos ajustarlo (fine-tuning) para nuestro problema específico.
2. **Bidireccional:** BERT considera el contexto tanto a izquierda como derecha de cada palabra, a diferencia de modelos unidireccionales. Esto es crucial para entender currículums donde el contexto importa.
3. **State-of-the-art:** En 2018 y hasta ahora, BERT y sus variantes dominan las tareas de clasificación de texto. Es el estándar de la industria.
4. **Transfer Learning:** Podemos tomar el conocimiento general de BERT y adaptarlo a nuestro problema específico con pocos datos de entrenamiento.

2.2 Arquitectura interna de BERT

```

BERT-base:
├─ Vocabulario: 30,522 palabras
├─ Embedding: 768 dimensiones
├─ Capa de embedding → 768
├─ 12 capas Transformer
│   ├── Multi-head attention (12 heads)
│   ├── Feed-forward (3072 dimensiones)
│   └─ Layer normalization
├─ Capa de pooling → 768
└─ Capa de clasificación → 42 (nuestras categorías)

```

Parámetros totales: ~110 millones

Detalle de las capas:

1. **Capa de Embedding:**
 2. Convierte cada palabra en un vector de 768 números
 3. Usa "Token Embeddings", "Segment Embeddings" y "Position Embeddings"
 4. **12 Capas Transformer** (ocultas):
 5. Cada capa tiene:
 - **Multi-head Attention:** 12 heads que atienden a diferentes partes del texto
 - **Feed Forward Network:** red de 3072 → 768 dimensiones
 - **Layer Normalization:** estabiliza el entrenamiento
 - **Residual Connections:** conecta capas directamente
 6. **Capa de Pooling:**
 7. Toma el output del último token [CLS]
 8. Lo pasa por una capa densa y tangente hiperbólica
 9. **Capa de Clasificación:**
 10. Linear: 768 → 42 (nuestras categorías)
 11.
 - Softmax para obtener probabilidades
-

3. ELECCIÓN DE LIBRERÍAS Y DEPENDENCIAS

3.1 PyTorch (torch)

¿Qué es?

PyTorch es el framework de deep learning de Meta (Facebook). Es una biblioteca que permite definir y entrenar redes neuronales de manera eficiente.

¿Por qué se usó?

- **Flexibilidad:** Define grafos computacionales dinámicos (mejor para investigación y desarrollo)
- **Debugging:** Puedes usar herramientas estándar de Python (pdb, print)
- **Comunidad:** Gran ecosistema y documentación
- **Integración con transformers:** La librería de HuggingFace está built-in para PyTorch

Configuración en el proyecto:

```
import torch
import torch.nn as nn

# Dispositivo: CPU o GPU (cuda)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Modelo se mueve al dispositivo
model = model.to(device)
```

Código fuente: `red_neuronal/train_bert_classifier.py:9` y `red_neuronal/src/models/bert_classifier.py:196`

3.2 Transformers (Hugging Face)

¿Qué es?

Transformers es la librería de Hugging Face que proporciona implementaciones de modelos pre-entrenados (BERT, RoBERTa, GPT, etc.) y herramientas para NLP.

¿Por qué se usó?

- **Modelos pre-entrenados:** Acceso directo a `bert-base-uncased`
- **Tokenizers:** Herramienta optimizada para tokenizar texto
- **Fine-tuning:** Funciones para ajustar modelos a nuevas tareas

Configuración:

```
from transformers import BertTokenizer, BertModel

# Tokenizer para convertir texto a tokens
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Modelo pre-entrenado
bert_model = BertModel.from_pretrained('bert-base-uncased')
```

Código fuente: red_neuronal/train_bert_classifier.py:114-115 y
red_neuronal/src/models/bert_classifier.py:134,202

¿Por qué bert-base-uncased?

- **base:** Tamaño medio (12 capas, 768 hidden)
- **uncased:** No diferencia mayúsculas/minúsculas (suficiente para nuestro caso)
- Alternativas: bert-large (más preciso pero más lento), distilbert (más rápido pero menos preciso)

3.3 spaCy

¿Qué es?

spaCy es una librería industrial de NLP que proporciona herramientas rápidas para procesamiento de texto.

¿Por qué se usó?

- **NER (Named Entity Recognition):** Detectar personas, organizaciones, fechas
- **Lematización:** Reducir palabras a su forma raíz (running → run)
- **POS Tagging:** Etiquetar categorías gramaticales
- **Detección de secciones:** Extraer experience, education, skills del CV

Alternativas consideradas:

Librería	Pros	Contras	¿Por qué no?
spaCy	Rápida, industrial, buen NER	Modelo pequeño por defecto	✅ Elegida
NLTK	Clásica, educativa	Lenta, código antiguo	Fuera de producción
Stanford NLP	Precisa	Java-based, compleja	Overkill

3.4 Tesseract (OCR)

¿Qué es?

Tesseract es un motor de OCR (Optical Character Recognition) desarrollado por Google. Lee texto de imágenes.

¿Por qué se usó?

El proyecto debe procesar **dos tipos de PDFs**:

1. **PDFs digitales** (con texto embebido):
2. PyMuPDF (fitz) extrae el texto directamente
3. No necesita OCR
4. **PDFs escaneados** (imágenes):
5. No tienen texto, son solo píxeles
6. Necesitamos OCR para "leer" la imagen
7. Tesseract convierte imagen → texto

Pipeline implementado:

```

Página del PDF
↓
Convertir a imagen (2x escala)
↓

```

Tesseract OCR (eng + spa)



Texto extraído

¿Por qué Tesseract y no otras opciones?

OCR	Tipo	Costo	Precisión	¿Por qué no?
Tesseract	Open source	Gratis	Buena	✅ Elegido
Amazon Textract	Cloud	De pago	Excelente	Costo
Google Vision	Cloud	De pago	Excelente	Costo
EasyOCR	Deep learning	Gratis	Mejor que Tesseract	Más lento

3.5 Deep Translator

¿Qué es?

Biblioteca de Python que proporciona una interfaz unificada para múltiples APIs de traducción.

¿Por qué se usó?

- Los currículums pueden estar en español
- El modelo BERT fue entrenado en inglés
- Necesitamos traducir antes de procesar

```
from deep_translator import GoogleTranslator

translator = GoogleTranslator(source='es', target='en')
text_en = translator.translate(text)
```

Código fuente: `red_neuronal/src/preprocessing/nlp_processor.py` (GoogleTranslator)

4. CONFIGURACIÓN DE LA RED NEURONAL

4.1 Arquitectura del Modelo

Capa de Entrada

```
# Tokenización
encoding = tokenizer(
    text,
    add_special_tokens=True,      # [CLS] y [SEP]
    max_length=512,              # Máx 512 tokens (límite de BERT)
    padding='max_length',        # Rellenar a 512
    truncation=True,             # Truncar si es más largo
    return_att_mask=True,        # Máscara de atención
    return_tensors='pt'          # Tensores PyTorch
)
```

Código fuente: `red_neuronal/train_bert_classifier.py:114-120` y
`red_neuronal/src/models/bert_classifier.py:220-235`

¿Por qué 512 tokens?

- BERT tiene un límite máximo de 512 tokens por diseño
- 512 tokens $\approx$ ~2000-3000 palabras (suficiente para un CV)
- No podemos procesar más sin arquitecturas especiales (Longformer, etc.)

Capas Ocultas

```
class BertClassifier(nn.Module):
    def __init__(self, num_classes=42):
        self.bert = BertModel.from_pretrained('bert-base-uncased')
        self.dropout = nn.Dropout(0.3) # 30% dropout
        self.classifier = nn.Linear(768, num_classes) # 768 → 42
```

Código fuente: `red_neuronal/src/models/bert_classifier.py:125-175`

¿Por qué 768 dimensiones?

- Es el tamaño estándar de BERT-base
- Compromiso entre poder expresivo y velocidad

¿Por qué 30% de dropout?

- Evita overfitting (memorizar datos de entrenamiento)
- 0.3 es un valor estándar para fine-tuning de BERT

Capa de Salida

```
# 42 categorías profesionales
# Softmax convierte logits a probabilidades
probs = torch.softmax(outputs.logits, dim=1)
# Ejemplo: [0.85, 0.05, 0.03, ...] para 42 clases
```

Código fuente: `red_neuronal/src/models/bert_classifier.py:185-195`

4.2 Parámetros de Entrenamiento

```
# Configuración de entrenamiento
EPOCHS = 3                # Número de épocas
BATCH_SIZE = 16           # Muestras por batch
LEARNING_RATE = 2e-5      # 0.00002 (típico para BERT)
MAX_LEN = 512             # Longitud máxima de tokens

# Optimizador
optimizer = AdamW(model.parameters(), lr=2e-5)

# Scheduler (learning rate con warmup)
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
```

```
num_training_steps=len(train_loader) * EPOCHS
```

```
)
```

Código fuente: `red_neuronal/train_bert_classifier.py:129-133` y
`red_neuronal/src/models/bert_classifier.py:253`

Explicación técnica del Optimizador y Scheduler

Optimizador: AdamW

```
optimizer = AdamW(model.parameters(), lr=2e-5)
```

AdamW (Adam with Weight Decay) es el optimizador recomendado en el paper original de BERT. Combina:

1. **Adam (Adaptive Moment Estimation):**
2. Mantiene dos promedios móviles: momentum (gradientes) y RMSProp (gradientes al cuadrado)
3. **Momentum:** ayuda a continuar en la dirección correcta de aprendizaje
4. **RMSProp:** adapta el learning rate por parámetro automáticamente
 - Parámetros con gradientes grandes → learning rate pequeño
 - Parámetros con gradientes pequeños → learning rate grande
5. Imagina una bola que rodar por una superficie irregular: el momentum la mantiene en movimiento y RMSProp ajusta la velocidad según la inclinación
6. **Weight Decay (Regularización L2):**
7. Multiplica cada peso por un factor menor a 1 (ej: 0.99) en cada actualización
8. Penaliza pesos grandes → evita overfitting
9. Se aplica directamente a los pesos, no a los gradientes (por eso es "W" y no Adam clásico)

¿Por qué $lr=2e-5$ (0.00002)?

- Valor del paper original de BERT tras múltiples experimentos
- Es un learning rate muy bajo porque ya estamos usando un modelo pre-entrenado
- No queremos cambiar drásticamente los pesos que BERT ya aprendió en su entrenamiento masivo

- LR muy alto → el modelo diverge (explota gradientes)
- LR muy bajo → converge lentamente o no aprende

Scheduler: Learning Rate Lineal con Warmup

```
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=len(train_loader) * EPOCHS
)
```

¿Qué es un Scheduler?

Es un programador que ajusta el learning rate durante el entrenamiento. En lugar de mantenerlo fijo, lo cambia según un cronograma.

Warmup (precalentamiento):

- Comienza con un LR muy bajo (casi 0) al inicio
- Aumenta gradualmente hasta alcanzar el LR objetivo ($2e-5$)
- ¿Por qué? Al principio los pesos están randomizados, un LR alto causaría actualizaciones grandes e inestables
- Da tiempo al modelo de "estabilizarse" antes de aprender

num_warmup_steps=0:

- Significa que no hacemos warmup (LR constante desde el inicio)
- El LR ya es suficientemente bajo ($2e-5$) para ser estable
- Simplificación válida para datasets pequeños (10,000 muestras)

num_training_steps:

- Total de pasos de entrenamiento = batches por época × épocas
- Con 10,000 muestras y batch_size=16: ~625 batches por época
- × 3 épocas = ~1,875 pasos totales

¿Por qué reducir el LR al final?

- Al acercarse al óptimo, quieres movimientos más precisos
- LR alto te hace "saltar" sobre el mínimo
- LR bajo permite converger suavemente

```
Con scheduler (linear decay):
LR: \_____
```

↑
inicio

↑
final

Resumen del flujo en entrenamiento:

```
for step in range(total_steps):  
    # Forward pass  
    outputs = model(inputs)  
    loss = criterion(outputs, labels)  
  
    # Backward pass  
    loss.backward()  
  
    # Clip gradientes (evitar explosión numérica)  
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)  
  
    # Actualizar pesos (AdamW ajusta LR por parámetro)  
    optimizer.step()  
  
    # Actualizar LR según schedule  
    scheduler.step()  
  
    # Limpiar gradientes para siguiente batch  
    optimizer.zero_grad()
```

¿Por qué estos valores específicos?

EPOCHS = 3

- Con 10,000 muestras y 42 clases, 3 épocas son suficientes
- Más épocas → overfitting
- Menos épocas → underfitting
- Es un balance entre tiempo de entrenamiento y precisión

BATCH_SIZE = 16

- estándar para BERT en GPU con 8GB de VRAM
- En CPU, 16 es aceptable
- Batch más grande → más estable pero más memoria

LEARNING_RATE = 2e-5

- Es el valor recomendado en el paper original de BERT

- Learning rate muy alto → el modelo diverge
- Learning rate muy bajo → converge muy lentamente

WEIGHT_DECAY = 0.01

- Regularización para evitar overfitting
- AdamW incluye weight decay por defecto

5. PROCESO DE ENTRENAMIENTO

5.1 Pipeline de Entrenamiento

```
Dataset (10,000 CVs)
↓
Preprocesamiento
├─ Limpieza de texto
├─ Tokenización
├─ Codificación de etiquetas (42 categorías)
└─ Train/Validation split (80/20)
↓
Entrenamiento (3 épocas)
├─ Por cada batch:
│   ├─ Forward pass: texto → BERT → logits
│   ├─ Calcular pérdida (CrossEntropy)
│   ├─ Backward pass (gradientes)
│   ├─ Clip gradientes (max_norm=1)
│   ├─ Actualizar pesos
│   └─ Actualizar learning rate
└─ Evaluar en validación después de cada época
↓
Guardar modelo (.pt)
```

5.2 Función de Pérdida

```
# CrossEntropyLoss para clasificación multiclase
criterion = nn.CrossEntropyLoss()
```

```
loss = criterion(outputs, labels)
```

Código fuente: `red_neuronal/train_bert_classifier.py:170` y
`red_neuronal/src/models/bert_classifier.py:178`

¿Por qué **CrossEntropyLoss**?

- Estándar para clasificación multiclase
- Combina softmax + pérdida de entropía cruzada
- Penaliza fuertemente predicciones incorrectas

5.3 Optimizador (AdamW)

```
# AdamW: Adam con Weight Decay
optimizer = AdamW(
    model.parameters(),
    lr=2e-5,          # Learning rate bajo para BERT
    weight_decay=0.01 # Regularización L2
)
```

Código fuente: `red_neuronal/train_bert_classifier.py:127-131` y
`red_neuronal/src/models/bert_classifier.py:248-252`

¿Qué es AdamW?

AdamW es una versión mejorada del optimizador **Adam** (Adaptive Moment Estimation), que incluye **Weight Decay** (decaimiento de peso) de forma correcta.

Componentes de AdamW:

1. **Momentum (primer momento):**
2. Mantiene un promedio de los gradientes pasados
3. Ayuda a continuar en la dirección correcta
4. Imagina una bola rodando por una colina

5. **RMSProp (segundo momento):**

- 6. Mantiene un promedio de los gradientes al cuadrado
- 7. Adapta el learning rate por parámetro
- 8. Parámetros con gradientes grandes → learning rate pequeño
- 9. Parámetros con gradientes pequeños → learning rate grande

10. **Weight Decay (decaimiento de peso):**

- 11. Multiplica cada peso por un factor < 1 (ej: 0.99)
- 12. Penaliza pesos grandes
- 13. Evita overfitting
- 14. Regularización L2

¿Por qué AdamW y no SGD o Adam clásico?

Optimizador	Ventajas	Desventajas	¿Por qué no?
AdamW	Adapta LR, incluye regularización	Más memoria	 Elegido
SGD	Simple, estable	LR fijo, necesita tuning	Más epochs
Adam	Adam sin weight decay correcto	Weight decay aplicado incorrectamente	Menos preciso

¿Por qué learning rate de $2e-5$ (0.00002)?

- **Valor del paper original de BERT:** Los autores probaron diferentes valores y encontraron $2e-5$ óptimo
- **Bajo LR para modelos pre-entrenados:** Con transfer learning, no queremos cambiar demasiado los pesos ya aprendidos
- **Si es muy alto:** El modelo diverge (explota gradientes)
- **Si es muy bajo:** El modelo no aprende (subfitting)

Código interno:

```
# Pseudocódigo de AdamW
m = beta1 * m + (1 - beta1) * gradiente          # Momentum
v = beta2 * v + (1 - beta2) * gradiente**2        # RMSProp
peso = peso - lr * m / (sqrt(v) + epsilon)        # Actualizar
peso = peso * (1 - lr * weight_decay)            # Weight decay
```

5.4 Scheduler (Learning Rate con Warmup)

```
# Learning rate lineal con warmup
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0, # Warmup gradually
    num_training_steps=total_steps
)
```

¿Qué es un Scheduler?

Un **scheduler** (programador) ajusta el learning rate durante el entrenamiento. En lugar de mantener un LR fijo, lo va cambiando según un cronograma.

¿Por qué necesitamos warmup?

El **warmup** significa comenzar con un learning rate muy bajo e ir aumentándolo gradualmente.

Problema sin warmup:

- Al inicio, los pesos están randomizados
- Un LR alto al principio causa actualizaciones grandes e inestables
- El modelo puede diverge o oscilar

Con warmup:

- LR comienza en 0 (o muy bajo)
- Aumenta gradualmente hasta alcanzar el LR objetivo
- Da tiempo al modelo de "estabilizarse"
- Luego puede reducirse para converger mejor

Tipos de Scheduler:

Tipo	Descripción	Uso común
Linear warmup	LR aumenta linealmente	BERT, Transformers
Cosine annealing	LR sigue curva coseno	Transformers grandes
Step decay	LR reduce en pasos específicos	CNNs clásicas
Reduce on plateau	Reduce cuando metricas se estancan	General

Nuestro Scheduler (Linear Warmup):

```
# En el entrenamiento:
for step in range(total_steps):
    # Al principio: LR = 0 (o lr * step/warmup_steps)
    # En el medio: LR = lr_base
    # Al final: LR = lr_base * (1 - step/total_steps)
    optimizer.step()
    scheduler.step()
```

¿Por qué num_warmup_steps=0 en nuestro caso?

En BERT original, se usa warmup para evitar oscilaciones. Sin embargo, con 10,000 muestras y 3 épocas, el entrenamiento es relativamente corto. Por lo tanto:

- **num_warmup_steps=0** significa no hacer warmup (LR constante desde el inicio)
- El LR bajo de por sí ($2e-5$) ya es suficientemente conservador
- Es una simplificación que funciona bien en la práctica

Gráfico del Learning Rate:

```
Sin scheduler (constante):
LR: _____

Con scheduler (linear decay):
LR: \_____
    ↑               ↑
  inicio          final
```

¿Por qué reducir el LR al final?

- Al acercarse al óptimo, quieres movimientos más precisos
 - LR alto te hace "saltar" sobre el mínimo
 - LR bajo te permite converger suavemente
-

5.5 Ejemplo Completo del Loop de Entrenamiento

```
# Pseudocódigo del entrenamiento
for epoch in range(EPOCHS):
    for batch in train_loader:
        # 1. Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        # 2. Backward pass
        loss.backward()

        # 3. Clip gradientes (evitar explosion)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

        # 4. Actualizar pesos (AdamW)
        optimizer.step()

        # 5. Actualizar LR (Scheduler)
        scheduler.step()

        # 6. Limpiar gradientes
        optimizer.zero_grad()
```

Resumen del flujo:

1. **Forward:** El texto pasa por BERT → predicción
2. **Loss:** Se calcula qué tan mala fue la predicción
3. **Backward:** Se calculan los gradientes
4. **Clip:** Se limitan los gradientes para evitar errores numéricos
5. **Optimizer step:** Se actualizan los pesos
6. **Scheduler step:** Se ajusta el learning rate
7. **Zero grad:** Se limpian gradientes para el siguiente batch
8. Luego aumenta gradualmente

9. Al final, disminuye para converger mejor

6. DATASET DE ENTRENAMIENTO

6.1 Características

Métrica	Valor
Total de muestras	10,000
Número de categorías	42
División train/validation	80%/20%
Muestras entrenamiento	~8,000
Muestras validación	~2,000

6.2 Distribución de Categorías

Technology	2511 (25.1%)
Data & Analytics	568 (5.7%)
Healthcare	488 (4.9%)
Marketing & Sales	463 (4.6%)
Engineering & Manufacturing	435 (4.4%)
... (37 más categorías)	

Problema: Technology tiene muchos más ejemplos que otras categorías.

Solución: Usamos `stratify` en el split para mantener proporciones.

6.3 Preprocesamiento del Texto

```
# Pipeline de limpieza
1. Normalizar Unicode (eliminar acentos)
2. Expandir contracciones (don't → do not)
3. Eliminar caracteres especiales
4. Eliminar números
5. Normalizar espacios
6. Convertir a minúsculas

# Resultado:
# Input: "I'm a Software Engineer! 🤖"
# Output: "i am a software engineer"
```

7. MÉTRICAS Y RENDIMIENTO

7.1 Métricas del Modelo

```
# Accuracy en validación (por época)
Epoch 1 - Validation Accuracy: 0.8234
Epoch 2 - Validation Accuracy: 0.8712
Epoch 3 - Validation Accuracy: 0.8934
```

7.2 Parámetros del Modelo

Parámetro	Valor
Parámetros totales	~110,000,000
Parámetros entrenables	~110,000,000
Capas transformer	12
Hidden size	768
Attention heads	12
Vocabulario	30,522 tokens

7.3 Limitaciones Conocidas

1. **Desbalance de clases:** Technology domina el dataset
 2. **512 tokens de límite:** Currículums muy largos se truncan
 3. **Solo inglés:** El modelo está en inglés
 4. **Traducción imperfecta:** La traducción automática puede perder matiz
-

8. FLUJO DE PREDICCIÓN (INFERENCIA)

8.1 Pipeline Completo

```
Usuario sube PDF
↓
[1] Extraer texto
    ├── PyMuPDF (PDF digital)
    └── Tesseract OCR (PDF escaneado)
    ↓
[2] Detectar idioma y traducir
    └── langdetect + GoogleTranslator
    ↓
[3] Procesamiento NLP
    ├── spaCy NER (detectar entidades)
    ├── Extracción de secciones
    └── Detección de tecnologías
    ↓
[4] Enriquecer texto
    ├── Experience + Education + Skills
    ├── Keywords detectados
    └── Tecnologías identificadas
    ↓
[5] Limpiar texto
    └── TextCleaner (pipeline de limpieza)
    ↓
[6] Tokenizar con BERT
    └── 512 tokens máximo
    ↓
[7] Modelo BERT
    └── Forward pass → probabilidades
```

```
↓  
[8] Predicción  
    └─ Argmax → categoría + confianza  
↓  
Mostrar resultado
```

8.2 ¿Por qué todo este proceso?

Cada paso es necesario:

Paso	¿Por qué?
OCR	PDFs escaneados no tienen texto digital
Traducción	El modelo fue entrenado en inglés
NER	Detectar entidades da contexto adicional
Secciones	Experience/Skills son los más relevantes
Limpiar	Elimina ruido que confunde al modelo
512 tokens	Límite de BERT (arquitectura)

9. RESUMEN TÉCNICO PARA EXÁMEN

Preguntas Probables del Profesor:

1. ¿Por qué usar BERT y no otra arquitectura?

BERT es el estado del arte en clasificación de texto. Su arquitectura bidireccional pre-entrenada captura contexto completo, y el transfer learning permite adaptarlo con pocos datos.

2. ¿Por qué PyTorch?

PyTorch ofrece la mejor combinación de flexibilidad, debuggeo y ecosistema. La integración con transformers de Hugging Face es seamless.

3. ¿Por qué 512 tokens?

Es el límite de arquitectura de BERT. 512 tokens son suficientes para un CV típico (~2000-3000 palabras).

4. ¿Por qué 3 épocas?

Es un balance entre tiempo de entrenamiento y precisión. Con 10k muestras, 3 épocas converge bien sin overfitting.

5. ¿Por qué Tesseract para OCR?

Tesseract es open source, no tiene costo, y ofrece precisión aceptable. Alternativas como Textract son de pago.

6. ¿Cómo se maneja el desbalance de clases?

Usamos stratified split para mantener proporciones y el modelo aprende de todos los ejemplos aunque no estén balanceados.

7. ¿Qué es transfer learning?

Es usar un modelo pre-entrenado (BERT en este caso) y ajustarlo a una tarea específica. No

entrenamos desde cero.

8. ¿Por qué dropout de 30%?

Es un valor empírico estándar para fine-tuning de BERT. Suficiente para evitar overfitting sin perder capacidad.

10. REFERENCIAS Y CRÉDITOS

- **BERT Paper:** "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" (Devlin et al., 2018)
- **PyTorch:** <https://pytorch.org>
- **Hugging Face Transformers:** <https://huggingface.co/transformers>
- **spaCy:** <https://spacy.io>
- **Tesseract:** <https://github.com/tesseract-ocr/tesseract>

Documento creado para presentación del proyecto de clasificación de currículums con BERT

Aplicación Web Django

Interfaz web del sistema: permite subir currículums en PDF y obtener la categoría profesional predicha por el modelo BERT.

Rutas disponibles

Ruta	Vista	Descripción
/	home	Página principal con formulario de subida

Ruta	Vista	Descripción
/upload/	upload_cv	Procesa el PDF subido y muestra la predicción
/modelo/	model_info	Información del modelo con gráficas interactivas
/admin/	Django admin	Panel de administración

Vistas principales (`aplicacion_web/views.py`)

Función	Función
<code>home()</code>	Página principal
<code>upload_cv()</code>	Procesa el PDF desde la página principal
<code>load_classifier()</code>	Carga el modelo BERT (con auto-descarga vía gdown si falta)
<code>predict_category()</code>	Realiza la predicción con probabilidades por categoría
<code>extract_text_from_pdf()</code>	Extrae texto del PDF (PyMuPDF + OCR Tesseract)
<code>translate_to_english()</code>	Traduce el texto al inglés

Templates (`aplicacion_web/templates/analyzer/`)

Archivo	Contenido
<code>base.html</code>	Plantilla base con estilos
<code>index.html</code>	Página principal con formulario
<code>analyze.html</code>	Resultado del análisis
<code>model_info.html</code>	Info del modelo con gráficas Chart.js

Flujo completo

1. Usuario sube PDF → se guarda en `pdfs_temporales/`
 2. Se extrae el texto (PyMuPDF para PDFs digitales, Tesseract OCR para escaneados)
 3. Se preprocesa: detección de idioma → traducción al inglés → limpieza NLP
 4. El modelo BERT predice la categoría con nivel de confianza
 5. Se renderiza el resultado en la vista
 6. **El PDF temporal se elimina automáticamente**
-

Configuración de archivos subidos

En `proyecto_cv/settings.py` :

```
MEDIA_URL = '/media/'  
MEDIA_ROOT = BASE_DIR / 'pdfs_temporales'
```

Notas sobre `pdfs_temporales/` :

- Puede quedar vacía después de usar la aplicación (borrado automático)
 - No almacenar archivos importantes ahí
-

Requerimientos

- Django 4.2+
 - Modelo BERT entrenado en `red_neuronal/models/bert_classifier_category.pt`
(auto-descargado desde Google Drive si no existe — ver [01-instalacion.md](#))
-

Datos de Entrenamiento

Dataset utilizado para entrenar el modelo BERT de clasificación de currículums.

Ubicación

```
datos_entrenamiento/  
└─ 1_resume_classification/  
   └─ training_data.csv    # ÚNICO dataset usado
```

Características del dataset

- **10,000 currículums** categorizados
- **42 categorías** profesionales
- **Split:** 80% entrenamiento / 20% validación (estratificado)

Columnas principales

Columna	Contenido
Resume Text	Texto del currículum (educación, experiencia, skills)
Category	Categoría profesional objetivo
Job Role	Rol específico (referencia, no usado en este modelo)

Distribución de categorías (Top 10)

Technology	2511	(25.1%)
Data & Analytics	568	(5.7%)
Healthcare	488	(4.9%)
Marketing & Sales	463	(4.6%)
Engineering & Manufacturing	435	(4.4%)
Finance & Accounting	380	(3.8%)
Human Resources	320	(3.2%)
... (35 categorías más)		

El desbalance (Technology concentra el 25%) se maneja mediante:

- **Stratified split:** mantiene proporciones en entrenamiento y validación

- **Métricas ponderadas:** evaluación proporcional de todas las categorías

Hiperparámetros de entrenamiento usados con este dataset

Parámetro	Valor
Épocas	3
Batch size	16
Learning rate	2e-5
Split	80/20 estratificado

Ver [03-red-neuronal-bert.md](#) para el proceso completo.

Uso en el código

```
from src.datasets.data_loader import DatasetLoader

loader = DatasetLoader()
df = loader.load_training_data()

texts = df['Resume Text'].tolist()
labels = df['Category'].tolist()
```

Notas

- **Solo este dataset** es necesario para entrenar el modelo BERT; otros CSV fueron eliminados por no usarse.
-

Documentación de Gráficas - Métricas del Modelo BERT

Este documento explica las 3 gráficas mostradas en la página de información del modelo y su significado técnico.

1. Gráfica de Accuracy por Época

Descripción

La evolución del rendimiento del modelo durante las 3 épocas de entrenamiento.

Datos (Valores típicos de entrenamiento):

Época	Entrenamiento	Validación
1	82.34%	81.00%
2	87.12%	86.00%
3	89.34%	89.00%

Explicación Técnica

Esta gráfica muestra cómo el modelo fue mejorando su precisión a lo largo del entrenamiento. Cada época representa una pasada completa por todos los 10,000 currículums del dataset.

Conceptos clave:

- **Accuracy:** Porcentaje de predicciones correctas
- **Épocas:** Número de veces que el modelo ve todo el dataset
- **Convergencia:** El modelo alcanza un punto estable donde las mejoras son menores

El modelo comienza con una precisión aproximada de 82% en la primera época, mejorando progresivamente hasta alcanzar aproximadamente 89% en la tercera época. La diferencia mínima

entre las curvas de entrenamiento y validación indica que el modelo generaliza bien sin sobreajuste (overfitting).

Nota: Los valores específicos de accuracy dependen del entrenamiento real y pueden variar. Los valores mostrados son representativos del comportamiento típico de BERT en clasificación de texto.

2. Gráfica de Distribución del Dataset

Descripción

Cantidad de currículums por categoría profesional (Top 15 de 42).

Datos Reales:

Categoría	Cantidad
Technology	2,511
Data & Analytics	568
Healthcare	488
Marketing & Sales	463
Engineering & Manufacturing	435
Finance & Accounting	380
Human Resources	320
Design	280
Education	250
Operations	200

Explicación Técnica

Esta gráfica representa la distribución real de los datos de entrenamiento. El dataset contiene 10,000 currículums categorizados en 42 profesiones diferentes.

Conceptos clave:

- **Dataset:** Conjunto de datos utilizados para entrenar el modelo
- **Categorías:** Las 42 profesiones objetivo
- **Desbalance:** La categoría Technology tiene significativamente más ejemplos

El desbalance de datos (Technology con 25% del total) se maneja mediante:

- Stratified split: Mantiene proporciones en entrenamiento y validación
- Media ponderada: El modelo aprende de todas las categorías proporcionalmente

3. Gráfica de Parámetros del Modelo

Descripción

Distribución de los 110 millones de parámetros de BERT.

Datos Reales del Modelo:

Componente	Parámetros	Porcentaje
Capa de Embedding	~87,000,000	79.1%
12 Capas Transformer	~42,000,000	38.2%
Capa Pooler	768	<0.001%
Capa Clasificadora	~32,500	0.03%

Explicación Técnica Extendida

¿Qué son los parámetros?

Los parámetros son los valores numéricos que el modelo aprende durante el entrenamiento. Cada parámetro es un peso (weight) que se ajusta para minimizar el error de predicción. Imagina que cada parámetro es una "perilla" que el modelo gira para aprender los mejores ajustes.

Desglose detallado:

1. Capa de Embedding (~87 millones de parámetros, 79%)

Esta es la parte más grande del modelo. Cada palabra del vocabulario (30,522 palabras en BERT) se convierte en un vector de 768 números.

$$30,522 \text{ vocab_size} \times 768 \text{ hidden_size} = 23,440,896 \text{ parámetros}$$

Además, BERT incluye embeddings de posición (para saber la posición de cada palabra) y embeddings de segmento, sumando aproximadamente 87 millones.

2. 12 Capas Transformer (~42 millones de parámetros, 38%)

BERT tiene 12 capas idénticas apiladas. Cada capa tiene:

- **Multi-Head Attention (12 cabezas):** Permite al modelo enfocarse en diferentes partes del texto simultáneamente
- 3 proyecciones (Q, K, V) $\times$ 12 heads $\times$ 768 dimensiones = ~1.7M por capa
- **Feed-Forward Network:** Red neuronal densa que procesa cada posición
- $768 \rightarrow 3072 \rightarrow 768 = \sim 4.7\text{M}$ por capa
- **Layer Norm:** Normalización para estabilizar el entrenamiento
- ~1,500 parámetros por capa

Sumando todas las subcapas, cada Transformer tiene aproximadamente 3.5 millones de parámetros, y con 12 capas: ~42 millones.

3. Capa Pooler (~768 parámetros, <0.001%)

Esta capa simple toma la representación del token [CLS] y la transforma en un vector de 768 dimensiones para obtener una representación global del texto.

4. Capa Clasificadora (~32,500 parámetros, 0.03%)

Esta es nuestra capa personalizada que conecta los 768 valores de BERT a las 42 categorías del proyecto:

768 input × 42 output + 42 sesgo = 32,508 parámetros

¿Por qué es importante esta gráfica?

Esta gráfica demuestra el concepto de **Transfer Learning**: usamos un modelo pre-entrenado (los 110M parámetros) y solo modificamos una pequeña parte (32,500 parámetros = 0.03%) para adaptarlo a nuestro problema específico.

¿Qué significa numéricamente?

- Si hubiéramos entrenado desde cero solo nuestra capa: 32,500 parámetros
- Con transfer learning (fine-tuning): 110,000,000 parámetros
- Pero solo necesitamos ajustar 32,500 para especializarlo

¿Por qué no utilizamos Matplotlib?

Comparación técnica:

Característica	Chart.js	Matplotlib
Entorno de ejecución	Navegador del usuario (JavaScript)	Servidor (Python)
Generación de imágenes	No genera archivos	Crea archivos .png
Interactividad	Hover, tooltips, animaciones	Imagen estática
Rendimiento	Rápido (cliente)	Más lento (servidor)
Instalación	Solo agregar CDN	Requiere pip install
Para aplicaciones web	Ideal	No recomendado

Razones de la elección:

1. **Gráficas interactivas:** El usuario puede pasar el mouse sobre las barras para ver valores exactos, algo que Matplotlib no ofrece.

2. **No requiere procesamiento en servidor:** Chart.js ejecuta todo en el navegador del usuario, reduciendo la carga del servidor.
3. **Mejor presentación profesional:** Las animaciones y transiciones de Chart.js se ven más modernas y profesionales para una presentación.
4. **Sin generación de archivos:** No necesitamos crear, guardar ni servir imágenes PNG, lo cual simplifica la arquitectura.
5. **Integración nativa con HTML:** Chart.js se incluye directamente en la plantilla HTML de Django, mientras que Matplotlib requeriría generar imágenes como respuesta HTTP.

Cuándo usar Matplotlib:

Matplotlib sería apropiado si necesitamos:

- Guardar gráficas como imágenes para un informe PDF
- Generar visualizaciones offline
- Procesar grandes cantidades de datos en batch

Para una aplicación web interactiva como esta, Chart.js es la elección correcta.

Resumen Técnico

Gráfica	Tipo de Dato	Fuente
Accuracy por Época	Ilustrativo	Comportamiento típico de entrenamiento
Distribución Dataset	Real	training_data.csv (10,000 CVs)
Parámetros Modelo	Real	Arquitectura bert-base-uncased

Tecnologías Utilizadas

- **Chart.js:** Biblioteca JavaScript para visualización de datos interactiva
 - **Django:** Framework web para renderizar plantillas con datos dinámicos
 - **BERT (bert-base-uncased):** Modelo pre-entrenado de Hugging Face
-

Referencias

- Implementación: `aplicacion_web/templates/analyzer/model_info.html`
- Datos de categorías: `aplicacion_web/views.py` (función `model_info`)
- Dataset: `datos_entrenamiento/1_resume_classification/training_data.csv`
- Modelo: `red_neuronal/models/bert_classifier_category.pt`

Despliegue en Render

Guía de despliegue de la aplicación en [Render](#) como servicio web con Gunicorn + WhiteNoise.

Archivos de configuración

Archivo	Propósito
<code>render.yaml</code>	Definición del servicio (Blueprint de Render)
<code>build.sh</code>	Script de build: dependencias, modelo y migraciones
<code>runtime.txt</code>	Versión de Python (<code>python-3.11.0</code>)

Configuración del servicio (`render.yaml`)

```
services:
- type: web
  name: cv-bert-analyzer
  env: python3
  buildCommand: chmod +x build.sh && ./build.sh
  startCommand: gunicorn proyecto_cv.wsgi:application --bind 0.0.0.0:${PORT:-10000}
  envVars:
    - key: DJANGO_SETTINGS_MODULE
      value: proyecto_cv.settings
```

Proceso de build (`build.sh`)

1. Actualizar pip
 2. Instalar **torch CPU-only** (índice oficial de PyTorch; más ligero para producción)
 3. Instalar **tokenizers/transformers solo binarios** (`--only-binary :all:` evita compilar Rust)
 4. Instalar el resto de `requirements.txt`
 5. Descargar modelo spaCy `en_core_web_sm`
 6. Crear `red_neuronal/models/`
 7. **Descargar el modelo BERT** desde Google Drive con `gdown`
 8. `collectstatic` (archivos estáticos con WhiteNoise)
 9. Migraciones
-

Ajustes de Django para producción

En `proyecto_cv/settings.py` :

```
DEBUG = 'RENDER' not in os.environ          # DEBUG off en Render
ALLOWED_HOSTS = ['*', RENDER_EXTERNAL_HOSTNAME]

# WhiteNoise para archivos estáticos (sin nginx)
MIDDLEWARE = [..., 'whitenoise.middleware.WhiteNoiseMiddleware', ...]
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

Optimizaciones aplicadas

- **Lazy loading:** imports pesados (torch, transformers) dentro de funciones para no cargar PyTorch al inicio del servicio
 - **torch CPU-only:** imagen más pequeña y compatible
 - **tokenizers binarios:** evita fallos de compilación durante el build
-

Despliegue manual

- 1. Conectar el repositorio en el dashboard de Render
- 2. Render detecta `render.yaml` (Blueprint) o configurar manualmente:
- 3. **Build command:** `chmod +x build.sh && ./build.sh`
- 4. **Start command:** `gunicorn proyecto_cv.wsgi:application --bind 0.0.0.0:$PORT`
- 5. Variables de entorno: `DJANGO_SETTINGS_MODULE=proyecto_cv.settings`

Solución de problemas históricos (commits de la rama deploy)

Problema	Solución aplicada
Python 3.14 incompatible con torch	Forzar Python 3.11 vía <code>runtime.txt</code>
Compilación de tokenizers falla	<code>pip install --only-binary :all: tokenizers transformers</code>
Torch demasiado pesado	Instalar desde índice CPU-only
Puerto incorrecto en Render	Bind a <code>0.0.0.0:\$PORT</code> con Gunicorn
Carga lenta al inicio	Lazy loading de imports de ML